src/java.base/share/native/launcher/main.c::WinMain/main() //启动入口

→ src/java.base/share/native/libjli/java.c::JLI_Launch()

→ src/java.base/unix/native/libjli/java_md.c::LoadJavaVM()

动态加载JVM_DLL这个共享库，从中获取JNI_CreateJavaVM()，
JNI_GetDefaultJavaVMInitArgs()和JNI_GetCreatedJavaVMs()
三个函数的实现，其中JNI_CreateJavaVM()是JVM初始化的核心入口

→ src/java.base/unix/native/libjli/java_md.c::JVMInit()

→ src/java.base/share/native/libjli/java.c::ContinueInNewThread()

→ src/java.base/unix/native/libjli/java_md.c::CallJavaMainInNewThread()

→ src/java.base/share/native/libjli/java.c::JavaMain()

→ src/java.base/share/native/libjli/java.c::InitializeJVM()

→ src/java.base/share/native/libjli/java.c::LoadMainClass()

→ src/java.base/share/native/libjli/java.c::GetApplicationClass()

→ GetStaticMethodID(); //获取main方法

→ CallStaticVoidMethod(); //执行应用main方法

→ LEAVE()

→ src/hotspot/share/prims/jni.cpp::JNI_CreateJavaVM()

→ src/hotspot/share/prims/jni.cpp::JNI_CreateJavaVM_inner()

→ src/hotspot/share/runtime/thread.cpp::create_vm()

→ ThreadLocalStorage::init() //线程本地存储初始化

→ ostream_init() //输入输出流

→ Arguments::process_sun_java_launcher_properties() //加载java额外参数

→ os::init() //OS初始化

→ create_vm_timer.start() //计时器开始

→ Arguments::init_system_properties()

//设置jvm和系统参数，java中调用System.getProperty()获取的值都在此方法中设置

→ JDK_Version_init() //JDK版本

→ Arguments::init_version_specific_system_properties()

//添加java.vm.specification.vendor,java.vm.specification.version和java.vm.vendor的值

→ LogConfiguration::initialize() //日志

→ HOTSPOT_VM_INIT_BEGIN() //虚拟机初始化开始

→ ostream_init_log() //输出流日志

→ Arguments::init_libraries_at_startup() //依赖库文件

→ Arguments::init_agents_at_startup() //代理

→ vm_init_globals() //初始化全局数据结构等

→ ObjectMonitor::Initialize() //java监视器

→ init_globals()

→ cache_global_variables() //缓存全局变量

→ JvmtiExport::enter_early_start_phase();

→ JvmtiExport::post_early_vm_start() // Notify JVMTI agents that VM has started (JNI is up)

→ initialize_java_lang_classes() //初始化java语言包

→ quicken_jni_functions() //设置int等基本类型

→ HOTSPOT_VM_INIT_END() //初始化完成

→ AttachListener::vm_start() //启动监听器

→ CompileBroker::compilation_init_phase1() //编译器初始化

→ initialize_jsr292_core_classes() //核心类初始化

→ call_initPhase2() //模块系统初始化

→ call_initPhase3() //类加载器初始化

→ SystemDictionary::compute_java_loaders() // cache the system and platform class loaders

→ ClassLoader::initialize_module_path() //初始化模块路径

→ JVMCI::initialize_compiler() //初始化编译器

→ JvmtiExport::enter_live_phase()

→ create_vm_timer.end()

```
jint init_globals() {
  management_init();
  JvmtiExport::initialize_oop_storage();
  bytecodes_init();
  classLoader_init1(); //加载java依赖库，设置bootstrap加载路径
  compilationPolicy_init(); //设置编译策略
  codeCache_init();
  VM_Version_init();
  stubRoutines_init1();
  universe_init();
  AsyncLogWriter::initialize();
  gc_barrier_stubs_init();
  interpreter_init_stub();
  accessFlags_init();
  InterfaceSupport_init();
  VMRegImpl::set_regName();
  SharedRuntime::generate_stubs();
  universe2_init();
  javaClasses_init();
  interpreter_init_code();
  referenceProcessor_init();
  ...
}
```

```
jint universe_init() {
  initialize_global_behaviours();
  GCLogPrecious::initialize();
  GCConfig::arguments()->initialize_heap_sizes(); //设置heap大小
  Universe::initialize_heap();
  Universe::initialize_tlab();
  Metaspace::global_initialize(); //元空间初始化，包括GC和内存大小等
  MetaspaceCounters::initialize_performance_counters();
  ClassLoaderData::init_null_class_loader_data(); //在元空间添加bootstrap加载器信息
}
```

```
→ Universe::genesis() → SystemDictionary::initialize() → vmClasses::resolve_all() //按顺序加载vmClassMacros.hpp中定义的basic classes
  → vmClasses::resolve_through()                                                  //如java.lang.Object, java.lang.String, java.lang.Class等
    → resolve_until() {
        for (auto id : EnumRange<vmClassID>{start_id, limit_id}) { // (inclusive start, exclusive end)
          resolve(id, CHECK) → SystemDictionary::resolve_or_fail()
        }
      }
```

```
src/hotspot/share/runtime/thread.cpp::Threads::initialize_java_lang_classes()
→ initialize_class() //加载java.lang.System, java.lang.Thread, java.lang.OutOfMemoryError, java.lang.NullPointerException等
  → src/hotspot/share/classfile/systemDictionary.cpp::resolve_or_fail() //加载
  → InstanceKlass::cast(klass)->initialize() //初始化
→ call_initPhase1() //Phase 1 of the system initialization in the library, initialize java.lang.System
  → src/java.base/share/classes/java/lang/System.initPhase1()
```

```
Threads::initialize_jsr292_core_classes() {
  initialize_class(vmSymbols::java_lang_invoke_MethodHandle(), CHECK); //加载过程同initialize_java_lang_classes()
  initialize_class(vmSymbols::java_lang_invoke_ResolvedMethodName(), CHECK);
  initialize_class(vmSymbols::java_lang_invoke_MemberName(), CHECK);initSystemClassLoader
  initialize_class(vmSymbols::java_lang_invoke_MethodHandleNatives(), CHECK);
}
```

```
Threads::call_initPhase3()
  → src/java.base/share/classes/java/lang/System.initPhase3()
    → src/java.base/share/classes/java/lang/ClassLoader.initSystemClassLoader() {
        ClassLoader builtinLoader = getBuiltinAppClassLoader();
        Class.forName("java.system.class.loader",builtinLoader).newInstance();
      }
    → Thread.currentThread().setContextClassLoader()
```

```
ClassLoader.getBuiltinAppClassLoader()
  → return Classloaders.appClassLoader()
    → new AppClassLoader(PLATFORM_LOADER, ucp);
```

```
Class.forName(String name, boolean initialize, ClassLoader loader)
→ src/java.base/share/native/libjava/Class.c::Java_java_lang_Class_forName0()
  → src/hotspot/share/prims/jvm.cpp::JVM_FindClassFromCaller() → find_class_from_class_loader()
    → SystemDictionary::resolve_or_fail()
```

→ src/java.base/share/native/libjli/java.c::LoadMainClass()

→ src/java.base/share/native/libjli/java.c::GetLauncherHelperClass() //加载sun.Launcher.LauncherHelper

→ src/java.base/unix/native/libjli/java_md_common.c::FindBootStrapClass(env,"sun/launcher/LauncherHelper")

→ src/hotspot/share/prims/jvm.cpp::JVM_FindClassFromBootLoader()

→ src/hotspot/share/classfile/systemDictionary.cpp::resolve_or_null()

→ LauncherHelper.checkAndLoadMain()

→ loadMainClass()

→ ClassLoader scl = ClassLoader.getSystemClassLoader() //若没有配置java.system.class.loader，则使用AppClassLoader

→ Class.forName(cn, false, scl)

...→ AppClassLoader.loadClass()

→ validateMainClass() //make sure the main method is static and returns a void

```
//入参: env, mainClass, "main", "([Ljava/lang/String;)V"
jni.cpp::GetStaticMethodID(JNIEnv *env, jclass clazz, const char *name, const char *sig)
→ jmethodID get_method_id(JNIEnv *env, jclass clazz, const char *name_str, const char *sig, bool is_static, TRAPS) {
  //1.方法信息
  const char *name_to_probe = name_str;
  TempNewSymbol name = SymbolTable::probe(name_to_probe, (int)strlen(name_to_probe)); //main
  TempNewSymbol signature = SymbolTable::probe(sig, (int)strlen(sig));
  //2.初始化mainClass
  KlassHandle klass(THREAD,java_lang_Class::as_Klass(JNIHandles::resolve_non_null(clazz)));
  klass()->initialize(CHECK_NULL); //执行<clinit>，初始化类
  //3.查找main方法
  Method* m = klass->lookup_method(name, signature);
  return m->jmethod_id();
}
```

→ src/hotspot/share/classfile/systemDictionary.cpp::resolve_or_fail() //加载

→ resolve_or_null()

→ resolve_instance_class_or_null_helper()

→ resolve_instance_class_or_null()

→ load_instance_class() → load_instance_class_impl()

1). → src/hotspot/share/classfile/classLoader.cpp::load_class() //调用者未指定classLoader时，使用vm classLoader

→ open_stream() //获取.class文件流

→ src/hotspot/share/classfile/klassFactory.cpp::create_from_stream()

→ ClassFileParser::ClassFileParser() //创建解析器

→ src/hotspot/share/classfile/classFileParser.cpp::create_instance_klass()

→ src/hotspot/share/oops/instanceKlass.cpp::allocate_instance_klass() //在方法区分配内存区域

→ fill_instance_klass() //将.class文件中的信息填充到新分配的内存区域

→ src/hotspot/share/classfile/javaClasses.cpp::java_lang_Class::create_mirror()

→ find_or_define_instance_class() //记录已加载的类

2). → JavaCalls::call_virtual() //用指定的加载器，例如JDK16的systemClassLoader

→ AppClassLoader.loadClass() → BuiltinClassLoader.loadClassOrNull()

→ 1. findLoadedClass() //检查是否已加载，若没加载过，继续

→ 2. 如果能找到对应模块，交给负责该模块的加载器处理，否则执行以下双亲委派逻辑

→ 3. parent.loadClassOrNull() //委托给父类加载

→ 4. findClassOnClassPathOrNull() //父类加载失败时，尝试自己加载

```
BuiltinClassLoader.findClassOnClassPathOrNull()
→ ucp.getResource() //获取.class文件内容
→ ClassLoader.defineClass()
  → ClassLoader.c::Java_java_lang_ClassLoader_defineClass1() //classLoader传入的是.this
  → 或者 ClassLoader.c::Java_java_lang_ClassLoader_defineClass2()
    → jvm.cpp::JVM_DefineClassWithSource
      → jvm_define_class_common()
        → SystemDictionary.cpp::resolve_from_stream()
          → resolve_class_from_stream()
            → KlassFactory.cpp::create_from_stream()
              → find_or_define_instance_class()
```

```
ClassFileParser::ClassFileParser()
→ parse_stream() //【验证】Verification(文件格式+元数据)
  1. Check magic value, version numbers, constant pool size //
  2. Parse and check constant pool
  3. Check access flags, this class, super class...
→ post_process_parsed_stream()
  → SystemDictionary::resolve_super_or_fail() //加载父类
```

```
→ src/hotspot/share/oops/klass.cpp::operator new()
  → src/hotspot/share/memory/metaspace.cpp::allocate(){
      MetaWord* result = loader_data->metaspace_non_null()->allocate(word_size, mdtype);
      //InstanceKlass是分配在ClassLoader的Metaspace（元空间）的方法区中
    }
```

```
javaClasses.cpp::java_lang_Class::create_mirror() //创建InstanceKlass在Java层的Class对象
→ InstanceMirrorKlass::cast(vmClasses::Class_klass())->allocate_instance()
  → Universe::heap()->class_allocate() //heap区创建表示该类的对象
→ initialize_mirror_fields() //【准备】Preparation
  → do_local_static_fields() //遍历所有static的field，调用如下方法设置初始值
    → initialize_static_field() → mirror()->obj_field_put() //heap
  → set_class_data() //设置class.classData的值
```

```
SystemDictionary::find_or_define_instance_class()
→ find_or_define_helper()
  → 查找class_loader的dictionary，若找到直接返回
  → define_instance_class()
    → class_loader不为空时，调用addClass()方法，更新classLoader.classes，防止该类被垃圾回收
    → add_to_hierarchy()
    → update_dictionary(name_hash, k, class_loader) {
        ClassLoaderData *loader_data = class_loader_data(class_loader);
        Dictionary* dictionary = loader_data->dictionary();
        dictionary->add_klass(hash, name, k);
      }
    → InstanceKlass::eager_initialize()
```

```
InstanceKlass::eager_initialize()
→ link_class_impl() {
    ik_super->link_class_impl() // link super class
    interk->link_class_impl() // link all interfaces implemented by this class
    1. verify_code() //【验证】Verification(字节码)
      → ClassVerifier::verify_class()
        → verify_method()
    2. rewrite_class() //重写类
    3. link_methods()
      //Set up method entry points for compiler and interpreter
      //so the method is ready to be called from interpreter, compiler, and vtables
      → set_interpreter_entry()
      → make_adapters()
    4. initialize_vtable
    5. initialize_itable
  }
```